# Open Source Projects: Portfolio Review

Oct 18, 2026 · @Syed Jafri

## The short version

All 30 repositories in the Open Source Projects list are yours, so this is a code review of your portfolio. I cloned each one and ran every test suite I could. About 13 projects build and pass on a clean clone, about 5 have real code with a fixable problem, and about 12 are docs, scaffolding or empty stubs.

The biggest risk is credibility, not code quality. A reviewer who opens a stub, or spots a mismatch between your portfolio page and the code, may start doubting the strong projects too.

## Tier 1: verified working

These 14 repos build and pass their own tests on a fresh clone.

| Project | Actual stack | Size (lines) | Result |
|---|---|---|---|
| Call Intelligence Summarizer | Python | 11.4k | 405 passed |
| Audio-Reactive Shader Playground | TypeScript, Vite | 10.6k | 339 passed |
| Lead Enrichment Pipeline | Python | 10.3k | 137 passed (run from `v1/`) |
| NexusForge HIL Drone Swarm | Python | 7.7k | 111 passed |
| Quote & Proposal Builder | TypeScript, Vitest | 8.8k | 109 passed |
| Pipeline Forecasting Engine | Python | 7.8k | 82 passed |
| Physics Puzzle Platformer | C++, Godot | 7.6k | 69 solver cases, 0 failures |
| IoT Environmental Monitor | C firmware, JS decoder | 4.5k | All C suites pass; decoder 5/5 |

| Project | Actual stack | Size (lines) | Result |
|---|---|---|---|
| Custom STM32 Dev Board | C firmware | 8.1k | All suites pass |
| Systems Monitoring Dashboard | Go | 14.1k | All packages pass |
| Outbound Sequencer | Go | 11.7k | All packages pass, incl. race tests |
| DevOps Pipeline Orchestrator | Go | 20.1k | Passes; several packages untested |
| ECS Game Engine | C++, Vulkan | 8.9k | Builds; 3/3 pass |
| AI Behavior Tree Toolkit | C++ | 9.7k | Builds; 5/5 pass |

Lead with Outbound Sequencer (guardrails as the product, race tests), Pipeline Forecasting (point-in-time correctness), the Physics Platformer (deterministic replays) and Call Intelligence (largest suite, and the only repo with an outside merged PR).

## Tier 2: real code, but something's wrong

- **Microservices API Framework** won't build for anyone who clones it. Its `go.mod` replaces deps with `../../godeps/...` folders that exist only on your machine. Fix: delete those lines and run `go mod tidy`.
- **ML Inference Engine** (Rust/WASM): the Rust crate's 7 tests pass, but plain `pytest` shows 806 failures because fixtures must be generated with `make fixtures` first. Make those tests skip cleanly when fixtures are missing.
- **Lead Enrichment** has one test that fails unless run from `v1/`, because it loads `config/field-policy.yaml` relative to the working directory. Resolve the path from the file's own location.
- **FPGA Video Processing Core** has real Verilog RTL, a Python golden model and testbenches. Icarus Verilog wasn't installable here, so it is unverified, not failing.
- **DevOps Orchestrator** passes, but the `exec` provider, `verifier`, `verifier/http` and the fakes have no tests. Those are the riskiest parts of a deployment tool.

## Tier 3: documentation and design, little code

- **Ramp Enterprise Network** is a legitimate networking project: Cisco configs, VLAN/HSRP/ACL docs and a draw.io diagram. Its README is honest about that, so it's fine as is.
- **Raspberry Pi Private NAS** has good docs but only about 30 lines of shell scripts. Fair for an infrastructure write-up.
- **color-looks MVP** has a README showing a full tree (Metal backend, `third_party/`, `docs/testing.md`) over about 250 lines of scaffolding. Label it as a spec or roadmap.
- **creative-copilot (SENSEI)** is a roughly 500-line demo skeleton whose README promises emotion-aware grading and Adobe/Resolve/Blender hooks that aren't built yet.

## Tier 4: stubs

- **README + PDF only:** Generative Poster System, Motion Timeline Editor, Procedural Texture Baker, Variable Font Inspector, Rollback Netcode Layer, Procedural Dungeon Roguelike.
- **README only:** Geospatial Tile Server.
- **Broken upload:** looksmax-ai has 5 files in nonsense nesting, such as `backend/app/backend/app/services/backend/frontend/frontend/package.json`.

## Problems across the whole list

The portfolio README in `Projects-SJ` lists stacks that don't match the code. This is the most damaging issue and the easiest to fix.

| Project | README claims | Code actually is |
|---|---|---|
| DevOps Pipeline Orchestrator | TypeScript, Node, Redis | Go |
| Systems Monitoring Dashboard | React, InfluxDB, Python | Go |
| Lead Enrichment Pipeline | Go, Kafka, Salesforce | Python |
| Outbound Sequencer | TypeScript, Temporal, SendGrid | Go |

| Project | README claims | Code actually is |
|---|---|---|
| Physics Puzzle Platformer | Unity, C#, Steamworks, 40 levels | C++ with Godot, 5 levels |
| AI Behavior Tree Toolkit | Google Test | Custom test harness |
| Quote & Proposal Builder | Next.js, Prisma, Stripe, DocuSign | None of these in package files |
| 7 stub repos | Full feature descriptions | README only |

- **History is one bulk drop per project.** 24 repos were created on Sep 11, and most arrived in a single "Implement v1" commit of 5,000–20,000 lines. Reviewers will read that as generated or pasted-in code. If you used AI help, say so plainly; either way, show working history from now on.
- **Only 2 of 30 repos have a license** (color-looks and NexusForge). Without one the code isn't legally reusable, so it isn't open source yet.
- **CI is patchy.** Repos with good suites, such as Call Intelligence, Lead Enrichment, the STM32 board and most Go repos, have no workflow.
- **Inconsistent layout.** Code sits in `v1/` in some repos and at the root in others, and docs live in a folder named `1—Documentation` with an em dash, which git shows as escaped bytes.

## What to do, in order

- [ ] Fix the stack table in `Projects-SJ`, and remove the 8 stubs from this list or move them to a "Planned" list.
- [ ] Fix the Microservices `go.mod`, the Lead Enrichment config path and the ML engine's fixture handling.
- [ ] Add a LICENSE (MIT or Apache-2.0) and a GitHub Actions workflow to each Tier 1 repo.
- [ ] Pin your top 4–6 projects with honest READMEs and a one-command test run.

Scope: these results come from fresh-clone builds, test runs and README-versus-code checks. Security and deep design quality were not reviewed.